

Projeto de Simulação Dinâmica HIL (Hardware-in-the-Loop)

Interface Serial PC–G29, ESP32 DevKit V1 e ECU Speeduino 48-Way

Instituição:

Instituto Federal de Minas Gerais – Campus Betim

Autor:

William Las-Cazas Queiroz (0104566)

Gerente Técnico – Equipe Elétrica

Aplicação:

Validação Dinâmica de Calibração HIL em Malha Fechada (KTM Duke 390)

30 de outubro de 2026

Sumário

1 Introdução ao Simulador HIL e Recursos do Projeto

Este documento descreve a arquitetura e o guia completo de reprodução da bancada de simulação dinâmica orientada ao condutor (*Hardware-in-the-Loop* – HIL) desenvolvida para validação e calibração da ECU **Speeduino 48-Way**. Na arquitetura adotada, um Computador (PC) atua como *host* USB para leitura em tempo real de um conjunto de volante, pedais e câmbio **Logitech G29** através de uma interface gráfica em Python/Tkinter, transmitindo os comandos do piloto e os parâmetros físicos do veículo via comunicação Serial (UART) para o microcontrolador da bancada.

O núcleo da planta física é executado em um **ESP32 DevKit V1 (30 pinos)**, responsável por resolver numericamente a dinâmica longitudinal e rotacional da motocicleta **KTM Duke 390** (~ 373 cm^3, monocilíndrica, roda fônica 22-2) e converter as grandezas virtuais em sinais elétricos analógicos e digitais que excitam a ECU em tempo real. A malha de simulação é fechada interceptando os pulsos de injeção e ignição gerados pela Speeduino por meio de interrupções de hardware (ISR).

Repositórios Oficiais para Reprodução (Código-Fonte e Mapa Base)

Todo o código-fonte do firmware do ESP32, a interface gráfica em Python, o arquivo JSON de parametrização da motocicleta e o mapa base completo da ECU estão disponíveis publicamente nos endereços abaixo:

- **Repositório GitHub (Firmware ESP32, Interface Tkinter e Parâmetros):** `https://github.com/LasCazas/SimulacaoHil.git`
- **Mapa Base da ECU e Configurações do TunerStudio (Google Drive):** Clique aqui para acessar a pasta do Mapa Base no Google Drive (`https://drive.google.com/drive/folders/1bbK6THQXaumvtmB5yQu2AqLAyRcFz2Z0`)

Importante: O arquivo `.msq` hospedado no Google Drive já contém toda a configuração base da Speeduino 48-Way (decodificador de roda fônica 22-2, mapeamento de portas, calibração de sensores e tabelas iniciais de VE e ignição).

2 Lista de Materiais (BOM de Bancada)

Para montar e reproduzir o ambiente HIL em bancada sem a necessidade de motor ou atuadores indutivos reais, utilizam-se os seguintes itens:

- **1x Computador (PC) + Conjunto Logitech G29:** Executa a interface gráfica (`interface_g29.py`), lê volante, pedais (acelerador, freio, embreagem) e câmbio H/borboletas via USB e comunica-se com o ESP32 a 115200 bps.
- **1x Placa ESP32 DevKit V1 (Clássico – 30 pinos):** Executa a integração numérica da física do motor e do veículo, gera sinais analógicos/PWM/I2S e mede os pulsos de retorno da ECU.
- **1x Módulo DAC de Áudio I2S PCM5102A:** Dedicado exclusivamente à síntese senoidal contínua via DMA da roda fônica **22-2** (*Crank*) sem sobrecarregar a CPU principal.
- **1x Módulo Conversor de Nível Lógico Bidirecional (2 ou 4 canais – MOSFET BSS138):** Responsável por condicionar e converter com segurança as saídas lógicas de injeção e ignição da Speeduino para o nível de 3,3 V do ESP32 em bancada (sem bobina ou bico físico).
- **2x Filtros Passa-Baixas RC ($R = 10$ kΩ, $C = 1\,\mu$F):** Conectados às saídas nativas de 8 bits `DAC1` e `DAC2` do ESP32 ($f_c \approx 15{,}9$ Hz) para suavizar os degraus de tensão de TPS e MAP antes da leitura pelo ADC da ECU.
- **1x ECU Speeduino 48-Way + Fonte 12 V de Bancada:** Controlador sob teste alimentado pelo pino pós-chave (`B_A1`) e referenciado ao mesmo barramento de `GND` lógico.

3 Arquitetura Lógica e Modelagem Dinâmica

O simulador opera em três subsistemas acoplados em tempo real:

1. **Interface Humana (PC + G29):** Aquisita os eixos dos pedais e botões de marcha (1ª a 6ª, Neutro e Ré virtual de bancada), transmite os comandos a 30 Hz para o ESP32 e exibe a telemetria retornada a 20 Hz. Também envia dinamicamente pela Serial todos os parâmetros físicos do arquivo `moto_duke390.json`, dispensando regravação do firmware ao ajustar a planta.
2. **Planta Física Virtual (ESP32):** Integra as equações diferenciais do conjunto motor–embreagem–transmissão–veículo com passo interno de 0,5 ms:
   - **Dinâmica do Virabrequim:** A aceleração angular $\dot{\omega}_e$ resulta do balanço na

inércia do motor J_e:

$$J_e \frac{d\omega_e}{dt} = \tau_{\text{ind}}(\omega_e, \text{MAP}, \eta_\lambda) + \tau_{\text{lenta}} - \tau_{\text{atrito}}(\omega_e) - \tau_{\text{emb}} \tag{1}$$

onde τ_{ind} interpola a curva de torque a plena carga (WOT, pico de ~ 35 N $\cdot$ m a 7000 RPM) modulada pela pressão do coletor (`MAP`), com limitador de rotação em 10500 RPM e controle proporcional de marcha lenta ($\sim$ 1400 RPM).

- **Acoplamento de Embreagem e Câmbio:** O torque transmitido τ_{emb} varia progressivamente conforme o curso do pedal de embreagem e o escorregamento entre ω_e e a rotação refletida da roda através da redução primária (2,90), relação da marcha engatada e redução final (3,00).
- **Dinâmica Longitudinal e Sinal VSS Realista:** A velocidade linear v do veículo é integrada sobre a massa efetiva (moto + piloto + inércia das rodas), descontando arrasto aerodinâmico ($\frac{1}{2}\rho C_d A v^2$), resistência ao rolamento ($C_{rr} mg$) e força de frenagem do pedal de freio. Dessa forma, o sinal **VSS** é derivado diretamente da rotação da roda: ao pisar na embreagem ou engatar o Neutro em movimento, a frequência do VSS decai suavemente pela inércia do veículo, reproduzindo fielmente o comportamento real em pista.

3. **Controlador (Speeduino) e Malha Fechada:** A ECU lê os sensores simulados e comanda injeção e ignição. Quando a chave `use_inj_feedback` está ativa no JSON, o ESP32 compara a largura de pulso real medida no bico (`INJ-1-OUT`) com a massa estequiométrica admitida por ciclo, escalando a eficiência η_λ do torque gerado (simulando perda de torque por mistura pobre/rica ou corte de combustível).

4 Mapeamento do Chicote e Condicionamento de Sinais

Para manter os dois conversores analógicos internos de 8 bits do ESP32 (`GPIO 25` e `GPIO 26`) dedicados aos sinais de **TPS** e **MAP**, os sinais de relógio do barramento **I2S** do módulo PCM5102A foram alocados nos pinos **GPIO 27 (`BCK`)**, **GPIO 14 (`LCK/WS`)** e **GPIO 22 (`DIN`)**. A Tabela **??** resume todas as conexões elétricas da bancada.

Tabela 1: Mapeamento completo entre ESP32 DevKit V1, condicionamento e Speeduino 48-Way.

Sinal	**ESP32**	**Tipo**	**Conector ECU**	**Condicionamento / Observação**
TPS (Borboleta)	GPIO 25	DAC1 (0–3,3 V)	`B_C7 (TPS)`	Filtro RC (10 kΩ + 1 µF) p/ GND
MAP (Coletor)	GPIO 26	DAC2 (0–3,3 V)	`B_C8 (MAP-EXT)`	Filtro RC (10 kΩ + 1 µF) p/ GND
Crank 22-2 (BCK)	GPIO 27	I2S BCK	`B_B2 (CRANK-IN)`	Ligado ao pino `BCK` do PCM5102A
Crank 22-2 (LCK)	GPIO 14	I2S WS/LCK		Ligado ao pino `LCK` do PCM5102A
Crank 22-2 (DIN)	GPIO 22	I2S DATA		Ligado ao pino `DIN` do PCM5102A*
Velocidade (VSS)	GPIO 16	PWM LEDC	`B_A7 (VSS)`	Onda quadrada 3,3 V proporcional a v
Embreagem	GPIO 15	Saída Digital	`B_B7 (Clutch_in)`	Nível lógico indicando pedal acionado
Feedback Injetor	GPIO 4	ISR (`CHANGE`)	`G_A1 (INJ-1-OUT)`	Via Conversor de Nível Lógico (`Chan2`)
Feedback Ignição	GPIO 5	ISR (`RISING`)	`G_B1 (IGN-1-OUT)`	Via Conversor de Nível Lógico (`Chan1`)
Referência GND	GND	Terra Comum	`B_A8` / `B_B3`	Unificar GND do ESP32, PCM5102A e ECU

*No módulo PCM5102A, manter o pino `SCK` aterrado ao `GND`, alimentar `VIN` em 5 V e conectar a saída de áudio analógica (`L`) em `B_B2` e o terra de áudio em `B_B3 (VR1-)`.

4.1 Condicionamento de Retorno com Conversor de Nível Lógico (Bancada sem Bobina e sem Bico Físicos)

Em um motor real, o acionamento direto de um bico injetor solenoide ou do primário de uma bobina gera picos indutivos reversos (*flyback*) de dezenas a centenas de volts, o que obrigaria o uso de optoacopladores rápidos ou circuitos de grampeamento dedicados. Contudo, **como na bancada de simulação HIL não conectamos um bico injetor nem uma bobina de ignição reais**, não há carga indutiva nem transientes de alta tensão nas saídas da ECU.

Nessas condições de bancada, um **Módulo Conversor de Nível Lógico Bidirecional** comercial baseado em transistores MOSFET (como o BSS138 com resistores de

pull-up integrados de 10 kΩ, ilustrado na Figura **??**) é perfeitamente capaz de realizar toda a interface elétrica de retorno entre a Speeduino e os pinos de 3,3 V do ESP32 (que **não** são tolerantes a 5 V ou 12 V):

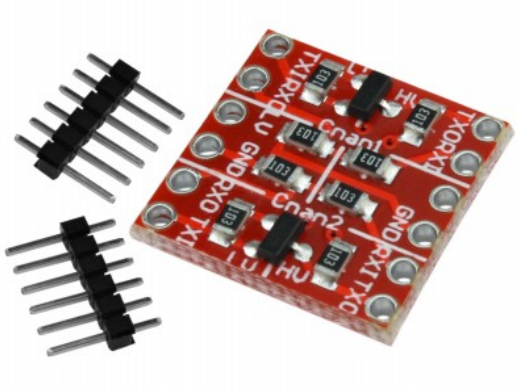


Figura 1: Módulo Conversor de Nível Lógico Bidirecional de 2 canais (`HV` ↔ `LV`) utilizado para leitura de injeção e ignição sem atuadores indutivos físicos.

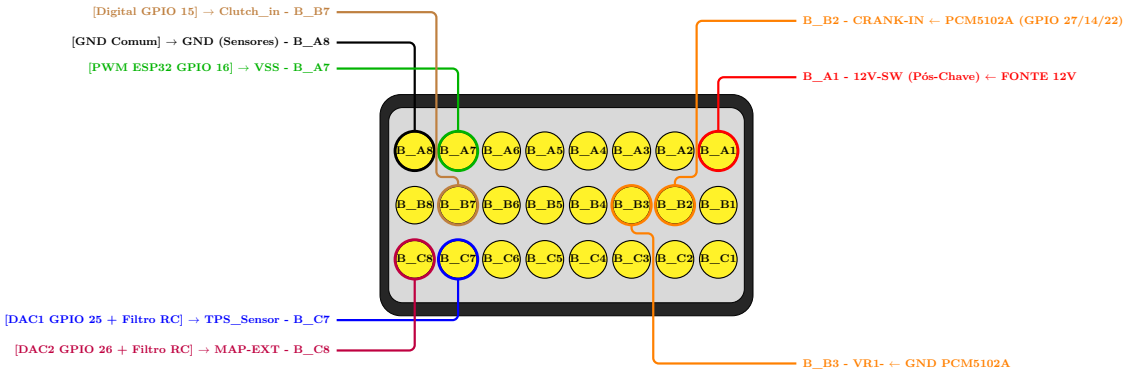
1. **Alimentação de Referência do Módulo:** O pino `HV` (*High Voltage*) do conversor é alimentado com 5 **V** (fornecidos pelo pino `5V/VIN` ou fonte lógica da bancada), o pino `LV` (*Low Voltage*) é conectado ao pino 3,3 **V** do ESP32, e os pinos `GND` de ambos os lados são interligados ao terra comum da ECU.

2. **Leitura do Bico Injetor (`G_A1 - INJ-1-OUT` → Canal 2 → `GPIO 4`):** A saída de injeção da Speeduino é do tipo *Low-Side* (dreno aberto / *open-drain*): o MOSFET de potência da ECU apenas aterra o pino ao `GND` durante o pulso de injeção e permanece flutuando quando desligado. Sem o bico físico puxando a linha para +12 V, o resistor de *pull-up* de 10 kΩ já existente no lado `HV` do conversor assume o papel de carga fantasma segura (corrente inferior a 0,5 mA). Em repouso, a linha permanece em nível alto (3,3 V no lado `LV`); quando a ECU comanda a injeção e aterra `G_A1`, o MOSFET do conversor conduz instantaneamente e leva o `GPIO 4` a 0 V, permitindo medir o tempo de injeção em microssegundos.

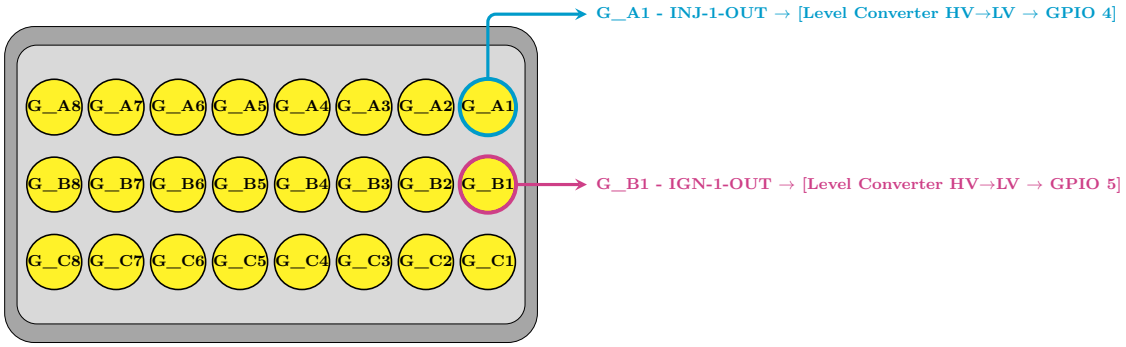
3. **Leitura da Ignição (`G_B1 - IGN-1-OUT` → Canal 1 → `GPIO 5`):** Configurada para bobina inteligente (*Smart Coil*), a Speeduino emite pulsos lógicos de 0–5 V. O sinal entra pelo terminal `HV` do Canal 1 e sai convertido de forma limpa em 0–3,3 V no terminal `LV` rumo ao `GPIO 5` do ESP32.

5 Diagrama Visual das Conexões Físicas

5.1 Envio de Sinais: Conector Preto (B)



5.2 Retorno de Sinais (Feedback): Conector Cinza (G)



6 Estrutura de Software, Protocolo e Guia de Reprodução

Para manter a documentação enxuta e sempre sincronizada com as evoluções do código, os arquivos executáveis encontram-se organizados no repositório GitHub (`https://github.com/LasCazas/SimulacaoHil.git`) conforme abaixo:

- `HIL_Esp/HIL_Esp.ino`: Firmware completo do ESP32 DevKit V1 (compatível com os pacotes Arduino-ESP32 `2.x` e `3.x`), integrando síntese DMA I2S da roda fônica 22-2, modelo dinâmico da motocicleta, saídas DAC/PWM e ISRs de injeção/ignição.
- `interface_g29.py`: Painel gráfico em Python (`tkinter` + `pygame` + `pyserial`) com **dois modos selecionáveis de operação**: *(i)* **Modo Volante Real (Logitech**

G29), com barras percentuais e leitura bruta de eixos/botões para calibração; e *(ii)* **Modo Simulado na Interface (Manual)**, com barras deslizantes (*sliders*) e entradas numéricas de 0% a 100% para os pedais e seletor clicável de marchas (`R`, `N`, `1..6`), permitindo operar e calibrar a ECU na bancada mesmo sem o G29 conectado.

- `moto_duke390.json`: Arquivo de parametrização documentado contendo 33 parâmetros escalares (inércia, cilindrada, vazão do bico, massa, aerodinâmica, embreagem, sensores), vetor de relações de marcha e curva de torque WOT.
- `config_g29.json`: Gerado e atualizado automaticamente ao fechar a interface gráfica, salvando a porta COM selecionada, o modo de controle ativo (`g29` ou `manual`), o mapeamento de eixos e as inversões dos pedais.
- `CodigoEsp/CodigoEsp.ino`: Código base original de bancada (geração isolada da roda fônica 22-2 via potenciômetros), mantido como referência histórica.

6.1 Protocolo de Comunicação Serial (115200 bps, 8N1)

A troca de mensagens em texto ASCII terminado por quebra de linha (`\n`) segue três formatos:

Tabela 2: Mensagens do protocolo Serial entre PC e ESP32.

Direção / Taxa	**Formato da Linha ASCII**	**Descrição dos Campos**
PC → ESP32 (30 Hz)	`thr,clu,gear,brk\n`	`thr`, `clu`, `brk` $\in [0, 255]$ (pedais); `gear` $\in [0, 7]$ (`0`=Neutro, `1..6`=Marchas, `7`=Ré).
PC → ESP32 (Conexão)	`P,nome,valor\n`	Atualiza em RAM qualquer parâmetro do `moto_duke390.json` (incluindo `g1..g6` e pontos `tqr`/`tqn`).
ESP32 → PC (20 Hz)	`rpm,vss,inj,ign,gr,kmh,tq,map,ff,ok\n`	RPM, VSS (Hz), pulso inj. (μs), idade da centelha (ms), marcha, veloc. ($\times$10 km/h), torque ($\times$10 N $\cdot$ m), MAP ($\times$10 kPa), fator comb. ($\times$100) e status da ECU (`0`/`1`).

6.2 Passo a Passo para Reproduzir o Experimento

1. **Configurar a ECU Speeduino no TunerStudio:** Baixe o projeto/mapa base (`.msq`) na pasta do Google Drive e carregue-o na Speeduino 48-Way pelo TunerStudio. Como os DACs nativos do ESP32 operam até 3,3 V (e não 5,0 V), realize a calibração de fim de escala do **TPS** no TunerStudio pisando 0% e 100% no pedal com o ESP32 conectado, e garanta que a faixa do sensor MAP e a vazão de bico coincidam com os valores de `moto_duke390.json`.

2. **Gravar o Firmware no ESP32 DevKit V1:** Abra `HIL_Esp/HIL_Esp.ino` na Arduino IDE (com o pacote oficial *esp32 by Espressif Systems* instalado), selecione a placa **ESP32 Dev Module** e faça o upload via USB.

3. **Montar as Conexões de Bancada:** Realize as ligações conforme a Tabela **??** e a Seção 5, certificando-se de instalar os filtros RC nos pinos 25 e 26, aterrar o pino `SCK` do PCM5102A e alimentar o Conversor de Nível Lógico com `HV = 5V`, `LV = 3.3V` e `GND` comum.

4. **Preparar o Ambiente Python e Executar a Interface:** Clone o repositório, crie/ative o ambiente virtual e instale as dependências:

```
git clone https://github.com/LasCazas/SimulacaoHil.git
cd SimulacaoHil
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python interface_g29.py
```

5. **Calibrar o G29 e Operar a Simulação:** Com o volante Logitech G29 conectado ao PC (com o Logitech G HUB aberto), selecione a porta `COM` do ESP32 e clique em **Conectar**. Após 2 s, todos os parâmetros de `moto_duke390.json` serão enviados automaticamente. Verifique se cada barra (Volante, Acelerador, Freio e Embreagem) responde ao respectivo pedal — ajustando se necessário o número do eixo ou a caixa `inv` (suas preferências ficarão salvas automaticamente ao fechar a janela). Para alterar características da moto ou ativar a malha fechada de combustível (`"use_inj_feedback": 1`), basta editar `moto_duke390.json` e clicar em **Reenviar parâmetros**.